

SiteSafety Tracker: Workplace Incident & Safety Management Platform

Progress Report & Draft Technical Documentation — Deliverable Milestone (Sept 23, 2026)

Course & Section:	CPE100L — Section A6	Submission Date:	September 23, 2026
Group Number:	Group 1	Application Type:	Web UI & Python RESTful API
Project Lead:	Andrei Santos (Lead Full-Stack)	Repository:	GitHub (SiteSafety-Tracker)

1. Project Overview

1.1 Problem Statement: Construction jobsites and industrial workspaces operate under high-risk conditions where physical safety hazards, scaffolding defects, equipment failure, and environmental dangers present severe risks to human life. Traditional safety reporting relies heavily on manual paper inspection sheets or disconnected verbal notifications, resulting in delayed corrective actions, lack of accountability, and zero real-time visibility for safety administrators.

1.2 Project Solution: *SiteSafety Tracker* is a full-stack Python and Web-based safety incident management system. It provides on-site field staff with an intuitive hazard ticket reporting interface (complete with image attachments, priority categorization, and location tagging) while empowering managers with live incident investigation boards, inspection scheduling engines, and automated audit logging.

1.3 Target Users:

- **Field Personnel / Safety Officers (Staff Role):** Submit real-time hazard reports, monitor personal report histories, and review active safety bulletins.
- **Safety Directors / Project Managers (Manager Role):** Review pending incident tickets, conduct root cause assessments, resolve tickets, schedule safety audits, and export incident compliance CSV datasets.

1.4 Team Member Role Distribution (Group 1):

Team Member	Assigned Role	Deliverable Responsibilities
Andrei Santos	Project Lead & Backend Architecture	Flask REST API, SQLite database integration, session management, and domain models.
Group 1 Member 2	Frontend UI & Interaction Developer	HTML5 layout hierarchy, Vanilla CSS design tokens, responsive views, modal dialogs.
Group 1 Member 3	Database & Business Logic Engineer	Schema normalization, SQL triggers, seed data scripts, and CSV export functionality.
Group 1 Member 4	QA, Security & Documentation Specialist	Input validation, bcrypt password hashing, session audit logging, and documentation.

2. Syllabus Topic Checklist & Implementation Strategy

The project has been engineered to strictly address all required computational concepts outlined in the CPE100L curriculum:

Syllabus Requirement	Status	Concrete Code Implementation Strategy
----------------------	--------	---------------------------------------

Classes & Objects (Required)	IMPLEMENTED	Defined core domain classes (User, HazardReport, Inspection, NotificationAlert) in server/models.py featuring parameterized <code>__init__</code> constructors, instance methods, and dictionary serialization.
Data Structures (Required)	IMPLEMENTED	Utilizes standard Python structures: Lists for queued records and batch iterations; Dictionaries for JSON payloads and user sessions; Tuples for SQLite parameterized queries; Sets for distinct category filtering.
Data Storage Strategy (Required)	IMPLEMENTED	Relational SQLite database (sitesafety.db) with 7 relational tables, persistent file writes, transaction safety, and dynamic CSV data export (SiteSafety_Resolved_Hazards.csv).
Error & Exception Handling (Required)	IMPLEMENTED	Robust try-except blocks wrapped around all database queries, authentication validations, file I/O operations, and HTTP endpoints returning structured JSON error codes (400, 401, 404, 500).
User Interface Framework (Required)	IMPLEMENTED	Full-featured Web UI powered by HTML5, Vanilla CSS Design System, and asynchronous JavaScript (Fetch API) interacting seamlessly with Python's Flask backend server.
Inheritance & Abstraction (Deferred / Optional)	DEFERRED	Flat domain model architecture with public attributes and encapsulated serialization methods in accordance with milestone guidelines.

3. System Architecture & Data File Design

3.1 Object-Oriented Domain Model (UML Architecture):

Below is the UML Class representation of the core Python domain models defined in server/models.py:

```
+-----+ | User |
+-----+ | - id: Optional[int] |
| - username: str | | - email: str | | - role: str ('staff' | 'manager') | | - company: str | | - signup_time:
str | +-----+ | +
__init__(username, email, role, company, user_id, ...) | | + is_manager() -> bool | | + is_staff() -> bool | |
+ to_dict() -> Dict[str, Any] | | + hash_password(plain_password: str) -> str [static] | | +
verify_password(plain_password: str, hashed_pw: str) -> bool [static] |
+-----+
+-----+ | HazardReport |
+-----+ | - ticket: str (e.g.,
'ABC-2026-0923-42') | | - location: str | | - category: str | | - urgency: str ('Low' | 'Medium' | 'High' |
'Critical') | | - status: str ('Pending' | 'Resolved') | | - cause: str | | - photo: str (Base64 data URL) | |
- reporter: Dict[str, str] | | - resolved_date: Optional[str] |
+-----+ | + __init__(ticket,
location, category, date, time, urgency, status, ...) | | + mark_resolved(resolution_date: Optional[str]) ->
None | | + is_resolved() -> bool | | + to_dict() -> Dict[str, Any] |
+-----+
```

3.2 Database Schema Layout (SQLite - sitesafety.db):

Table Name	Primary Columns & Data Types	Purpose & Constraints
users	id (INT PK), username (TEXT), email (TEXT UNIQUE), password (TEXT), role (TEXT), company (TEXT), signup_time (TEXT)	Stores user profiles with bcrypt hashed passwords and role constraints ('staff' / 'manager').
hazards	id (INT PK), ticket (TEXT UNIQUE), location (TEXT), category (TEXT), urgency (TEXT), status (TEXT), cause (TEXT), photo (TEXT), reporter_name (TEXT), resolved_date (TEXT)	Stores workplace hazard incident reports, photographic evidence, and lifecycle resolution status.
inspections	id (INT PK), title (TEXT), location (TEXT), inspector (TEXT), date (TEXT), time (TEXT), status (TEXT)	Tracks site audit schedules and certified inspector assignments.
login_logs	id (INT PK), user_id (INT FK), username (TEXT), action (TEXT), status (TEXT), timestamp (TEXT), ip_address (TEXT)	Maintains security audit trails for logins, signups, and logouts.
notifications	id (INT PK), title (TEXT), time_label (TEXT), unread (INT), user_id (INT)	Delivers dynamic system alerts and ticket status change notices.

4. Final Project Features List

The following complete set of core features are implemented and will be demonstrated during the final course presentation:

- **1. Role-Based Authentication & Session Access Control:** Multi-role login supporting Staff and Manager personas with bcrypt cryptographic password protection and secure Flask server session cookies.
- **2. Hazard Incident Reporting Engine:** Comprehensive hazard filing interface with auto-generated tickets (e.g. ABC-2026-0923-XX), multi-tier urgency classification, location tags, and live photo attachment previews.
- **3. Interactive Manager Resolution & Investigation Board:** Tabular and card views allowing safety managers to inspect pending tickets, view photo evidence, and mark incidents as 'Resolved' with automated audit timestamps.
- **4. Scheduled Safety Inspection Calendar:** Dynamic scheduling workflow to assign certified inspectors to specific jobsite sectors with automatic synchronization into the pending hazards queue.
- **5. Live System Notification Feed:** Real-time notification dropdown and alerts informing personnel of new hazard reports, resolved incidents, and scheduled inspections.
- **6. Safety Contacts Directory:** Searchable emergency contact roster of site safety coordinators, medical teams, and compliance officers.
- **7. CSV Compliance Data Export:** One-click export enabling managers to download resolved hazard logs (SiteSafety_Resolved_Hazards.csv) for regulatory compliance.
- **8. Dual Execution Mode (Local & Cloud):** Seamless local execution via python main.py and cloud deployment capability via PythonAnywhere WSGI.

End of Deliverable Documentation — Prepared by CPE100L Section A6 Group 1.